

Getting Started with pgscatalogcolors

Contents

What ancestry acronyms are available?	1
What does my data frame need?	2
PGS odds ratio forest plot	2
Basic scatter plot	3
Bar chart (fill aesthetic)	4
Using <code>scale_color_manual()</code> directly	5
Checking colors programmatically	6
What if my labels have typos or inconsistent formatting?	6
What if my data uses full ancestry names instead of acronyms?	7

```
library(pgscatalogcolors)
library(ggplot2)
```

What ancestry acronyms are available?

The package maps **19 ancestry categories** defined by the PGS Catalog ancestry documentation. Call `pgscatalog_ancestry_info()` to see the full lookup table:

```
pgscatalog_ancestry_info()
#>   acronym          ancestry pop_group  color
#> 1   ABO             Aboriginal Australian  DIV #999999
#> 2   AAM African American or Afro-Caribbean  AFR #F9DA49
#> 3   AFR             African unspecified  AFR #F9DA49
#> 4   ASN             Asian unspecified  ASN #B25928
#> 5   CAS             Central Asian  ASN #B25928
#> 6   EAS             East Asian  EAS #68AD57
#> 7   EUR             European  EUR #377EB8
#> 8   MID             Greater Middle Eastern  MID #5DCBCF
#> 9   HIS             Hispanic or Latin American  HIS #D1352B
#> 10  AMR             Native American  DIV #999999
#> 11  NON             Not Reported  NON #BBBBBB
#> 12  OCE             Oceanian  DIV #999999
#> 13  OTH             Other  DIV #999999
#> 14  ADA             Other admixed ancestry  DIV #999999
#> 15  SAS             South Asian  SAS #984EA4
#> 16  SEA             South East Asian  ASN #B25928
#> 17  SAF             Sub-Saharan African  AFR #F9DA49
#> 18  MIE Multi-Ancestry including Europeans  MIE #E887BD
#> 19  MEE Multi-Ancestry excluding Europeans  MEE #EF8632
```

The columns are:

Column	Description
acronym	The exact string your data column must contain
ancestry	Full PGS Catalog ancestry label

Column	Description
pop_group	Broader population group abbreviation (1000G/HGDP-style)
color	Hex color assigned to that acronym

What does my data frame need?

Your data frame needs **one column whose values are ancestry acronyms** — the exact strings from the `acronym` column above (case-sensitive). The column can have any name; you point `ggplot2` to it with `aes(color = <your_column>)`.

```
# Minimal working example
df <- data.frame(
  sample_id = paste0("s", 1:6),
  pgs_score = c(0.42, -0.11, 0.87, 0.03, -0.55, 0.29),
  ancestry = c("EUR", "AFR", "EAS", "SAS", "HIS", "MID")
  #           ^-- must match acronyms exactly
)
df
#>   sample_id pgs_score ancestry
#> 1         s1      0.42      EUR
#> 2         s2     -0.11      AFR
#> 3         s3      0.87      EAS
#> 4         s4      0.03      SAS
#> 5         s5     -0.55      HIS
#> 6         s6      0.29      MID
```

The `ancestry` column values must be **character or factor** — numeric codes will not match. The column does **not** need to contain all 19 acronyms; only the ones present in your data are plotted.

PGS odds ratio forest plot

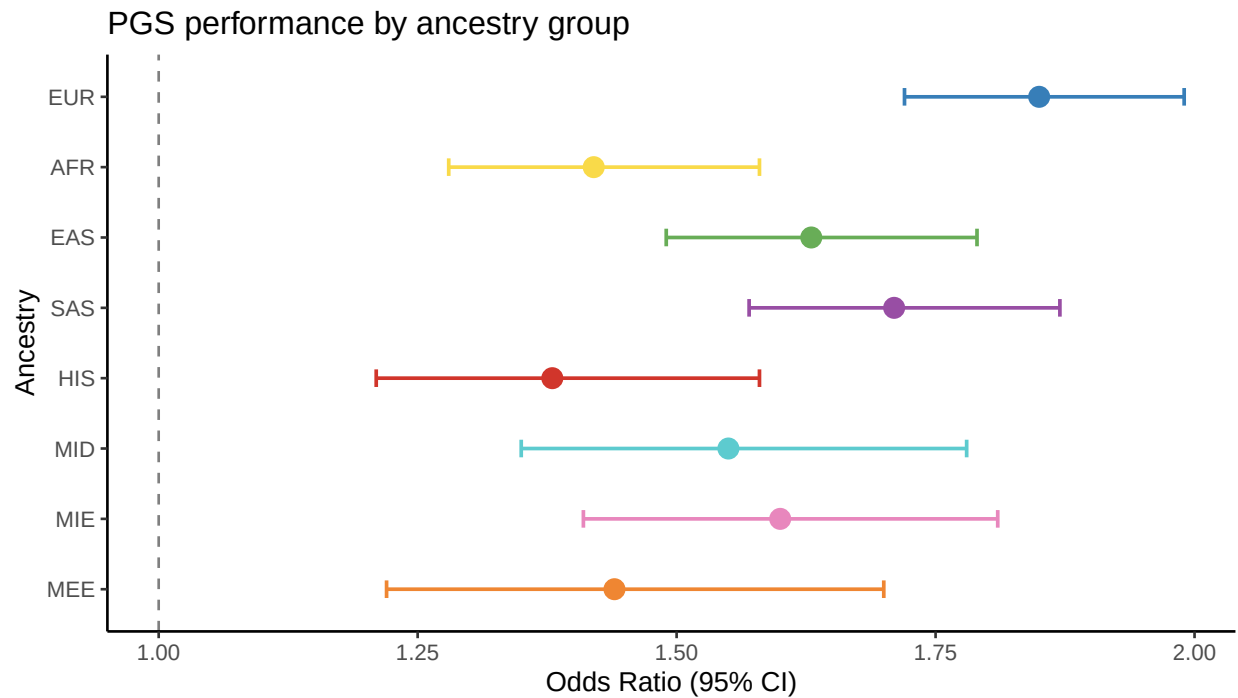
A common use case is plotting the odds ratio (OR) of a polygenic score across ancestry groups, with ancestry on the y-axis and the OR on the x-axis. Each point is coloured by ancestry using the PGS Catalog palette.

```
set.seed(42)
groups <- c("EUR", "AFR", "EAS", "SAS", "HIS", "MID", "MIE", "MEE")

forest_df <- data.frame(
  ancestry = factor(groups, levels = rev(groups)),
  OR       = c(1.85, 1.42, 1.63, 1.71, 1.38, 1.55, 1.60, 1.44),
  lower    = c(1.72, 1.28, 1.49, 1.57, 1.21, 1.35, 1.41, 1.22),
  upper    = c(1.99, 1.58, 1.79, 1.87, 1.58, 1.78, 1.81, 1.70)
)

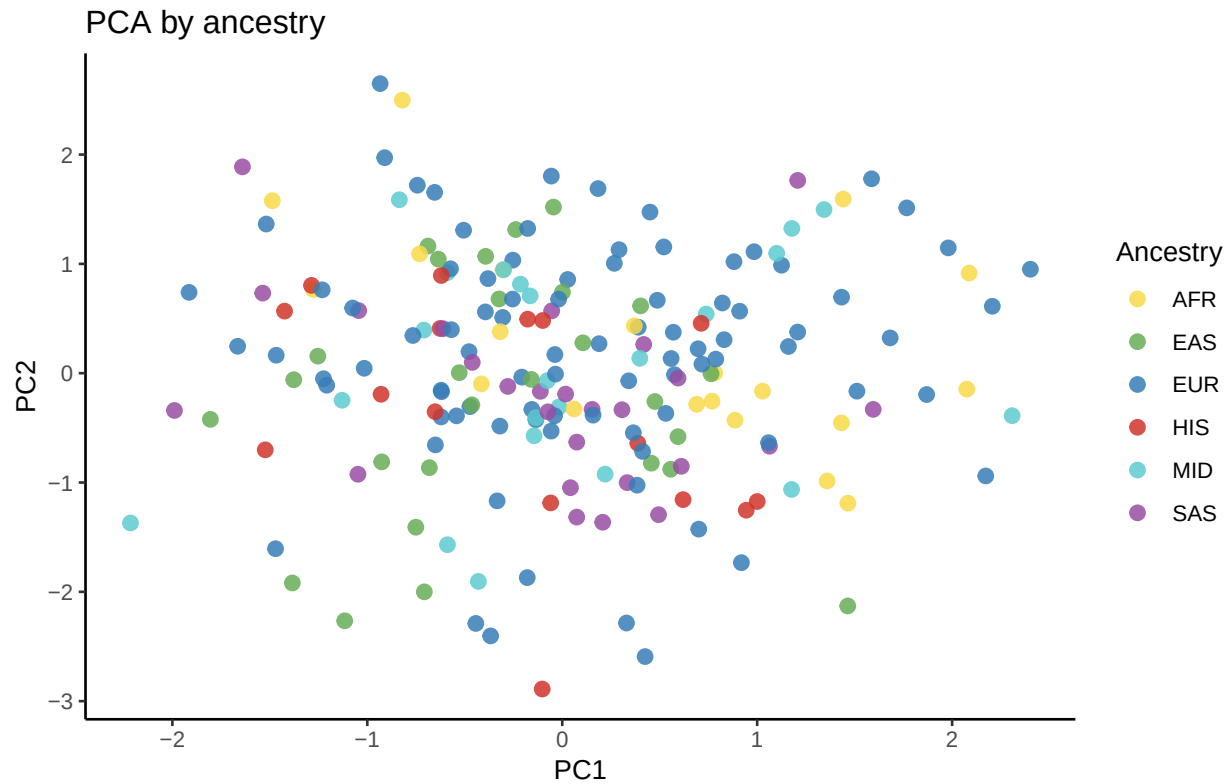
ggplot(forest_df, aes(x = OR, y = ancestry, color = ancestry)) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey50") +
  geom_errorbarh(aes(xmin = lower, xmax = upper), height = 0.25, linewidth = 0.7) +
  geom_point(size = 3.5) +
  scale_color_pgscatalog() +
  scale_x_continuous(breaks = seq(1, 2, 0.25)) +
  labs(
    x = "Odds Ratio (95% CI)",
    y = "Ancestry",
    title = "PGS performance by ancestry group"
```

```
) +  
theme_classic() +  
theme(legend.position = "none")
```



Basic scatter plot

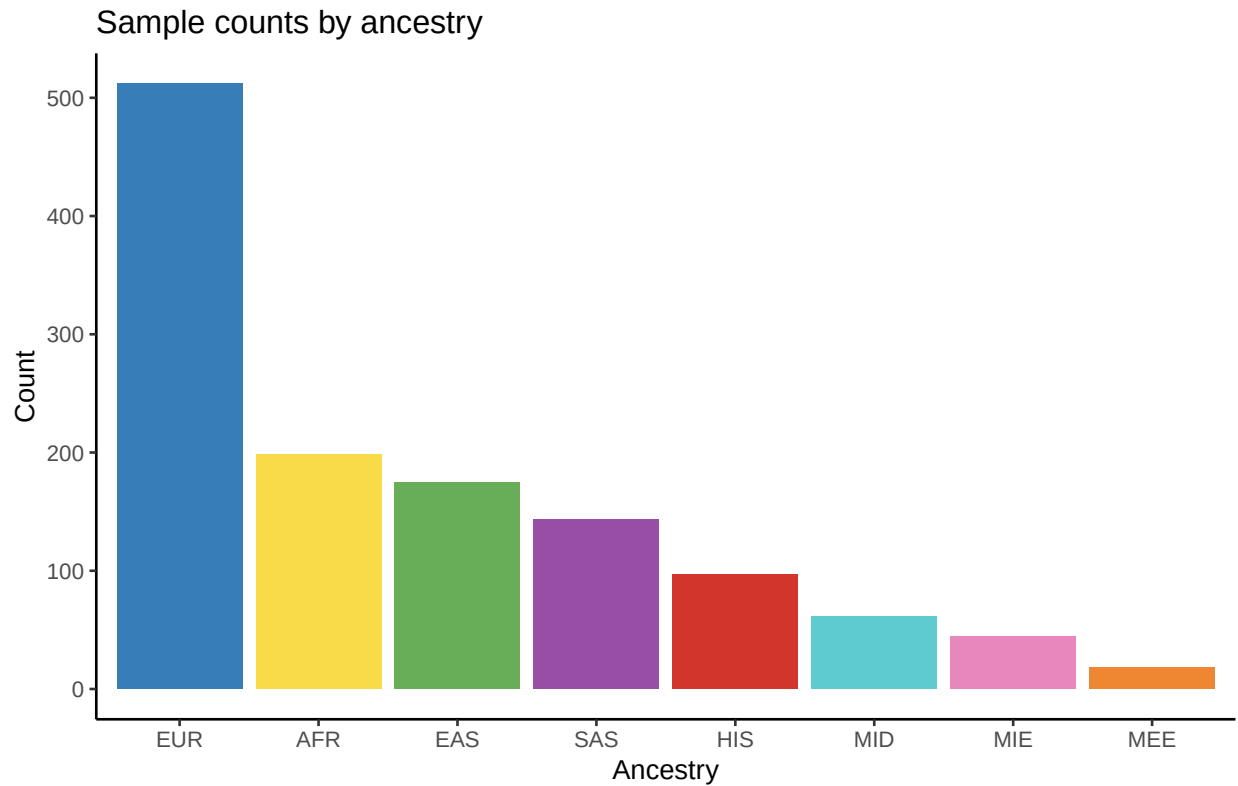
```
set.seed(1)  
n <- 200  
grp <- c("EUR", "AFR", "EAS", "SAS", "HIS", "MID")  
  
df <- data.frame(  
  PC1 = rnorm(n),  
  PC2 = rnorm(n),  
  ancestry = sample(grp, n, replace = TRUE,  
                    prob = c(0.40, 0.15, 0.15, 0.12, 0.10, 0.08))  
)  
  
ggplot(df, aes(PC1, PC2, color = ancestry)) +  
  geom_point(size = 2.5, alpha = 0.85) +  
  scale_color_pgs catalog() +  
  labs(title = "PCA by ancestry", color = "Ancestry") +  
  theme_classic()
```



Bar chart (fill aesthetic)

```
counts <- data.frame(
  ancestry = c("EUR", "AFR", "EAS", "SAS", "HIS", "MID", "MIE", "MEE"),
  n         = c(512, 198, 175, 143, 97, 61, 44, 18)
)

ggplot(counts, aes(reorder(ancestry, -n), n, fill = ancestry)) +
  geom_col() +
  scale_fill_pgscatalog() +
  labs(x = "Ancestry", y = "Count", title = "Sample counts by ancestry") +
  theme_classic() +
  theme(legend.position = "none")
```

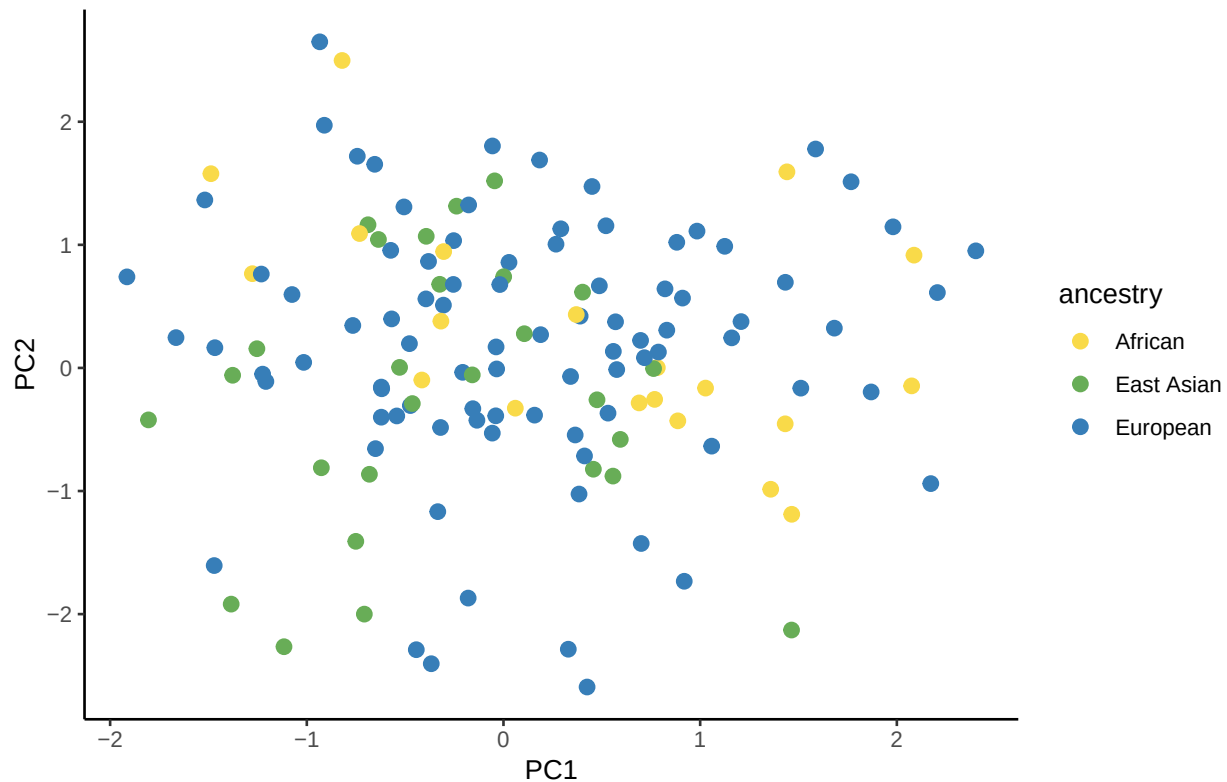


Using `scale_color_manual()` directly

If you need more control (e.g., custom legend labels), use `pgscatalog_scale_values()` to get only the colors for groups in your data:

```
my_groups <- c("EUR", "AFR", "EAS")

ggplot(df[df$ancestry %in% my_groups, ], aes(PC1, PC2, color = ancestry)) +
  geom_point(size = 2.5) +
  scale_color_manual(
    values = pgscatalog_scale_values(my_groups),
    labels = c(AFR = "African", EAS = "East Asian", EUR = "European")
  ) +
  theme_classic()
```



Checking colors programmatically

```
# Look up specific colors by acronym
pgscatalog_cols("EUR", "AFR", "EAS")
#>      EUR      AFR      EAS
#> "#377EB8" "#F9DA49" "#68AD57"

# Check whether your acronyms are all recognized before plotting
my_data_groups <- c("EUR", "AFR", "XYZ") # "XYZ" is not a valid acronym
pgscatalog_scale_values(my_data_groups) # warns about unknown acronyms
#> Warning in pgscatalog_scale_values(my_data_groups): Unknown acronyms (will use
#> grey): XYZ
#>      EUR      AFR      XYZ
#> "#377EB8" "#F9DA49" "#999999"
```

What if my labels have typos or inconsistent formatting?

Use `pgscatalog_match()` to fuzzy-match a column of messy ancestry labels to their PGS Catalog acronyms. Matching tries exact acronym, then exact full name, then edit-distance fuzzy matching — in that order.

```
messy <- c("EUR", "eur", "european", "East Asian", "east_asian", "EAST ASIAN")
pgscatalog_match(messy, quiet = TRUE)
#> [1] "EUR" "EUR" "EUR" "EAS" "EAS" "EAS"
```

Fuzzy matches (within `max_dist` edits, default 3) always produce a warning so you can verify them:

```
pgscatalog_match(c("Europeen", "sas", "Sub-Saharan African"))
#> Warning: Fuzzy matches accepted (verify these are correct):
```

```
#> "European" -> EUR
#> [1] "EUR" "SAS" "SAF"
```

Labels with no match within `max_dist` return NA with a warning:

```
pgscatalog_match(c("EUR", "martian", "unknown_ancestry"))
#> Warning: No match found (returned NA) for:
#> "martian"
#> "unknown_ancestry"
#> Call pgscatalog_ancestry_info() to see valid labels.
#> [1] "EUR" NA NA
```

The typical workflow is to call `pgscatalog_match()` once to add a clean acronym column, then pass that column to the `ggplot2` scales:

```
df_messy <- data.frame(
  sample_id = paste0("s", 1:6),
  pgs_score = c(0.42, -0.11, 0.87, 0.03, -0.55, 0.29),
  ancestry_raw = c("european", "African American or Afro-Caribbean",
    "east_asian", "South Asian", "HIS", "mid")
)

df_messy$ancestry <- pgscatalog_match(df_messy$ancestry_raw, quiet = TRUE)
df_messy[, c("ancestry_raw", "ancestry")]
#>
#> 1 european EUR
#> 2 African American or Afro-Caribbean AAM
#> 3 east_asian EAS
#> 4 South Asian SAS
#> 5 HIS HIS
#> 6 mid MID
```

What if my data uses full ancestry names instead of acronyms?

For exact full names, `pgscatalog_match()` handles this directly. For cases where you want to inspect or customise the mapping, you can also join against `pgscatalog_ancestry_info()` manually:

```
raw <- data.frame(
  sample_id = 1:3,
  ancestry_long = c("European", "African unspecified", "East Asian")
)

info <- pgscatalog_ancestry_info()
merged <- merge(raw, info[, c("ancestry", "acronym")],
  by.x = "ancestry_long", by.y = "ancestry")
merged
#>
#> 1 African unspecified 2 AFR
#> 2 East Asian 3 EAS
#> 3 European 1 EUR
```